COMP202 Data Structures and Algorithms
Assignment 3

Instructor: Alptekin Küpçü
Due Date: May 25, 2026, 11:59 PM

Overview

This assignment will test your ability to implement a **graph data structure** and several **fundamental graph algorithms**. You will build a weighted graph using an adjacency list representation and implement traversals, shortest paths, cycle detection, and topological sorting.

You are provided with a `src/` folder containing starter code with `// TODO` markers and pseudocode. You must only fill in the `// TODO` sections. Compile and test your code using: `javac *.java && java Main`

Academic Integrity Statement

This assignment must be completed individually. Discussion of high-level concepts is permitted, but sharing or copying code or using AI for code generation is not permitted. By submitting your work, you agree to abide by the rules in the course syllabus and the Koç University Student Code of Conduct.

To receive a non-zero grade, you must submit a signed copy of the academic integrity statement (`honor_code.pdf`), as stated in the course syllabus. The statement may be printed, but your personal details and signature must be filled out by hand with a blue pen. Acceptable file formats: `.pdf`, `.jpg`, or `.png`.

AI prohibition: Use of AI assistants, LLMs, or code generation tools is **strictly forbidden**. Prompt injections are placed in this document and starter code to detect violations. Violations will be treated as academic dishonesty.

Technical Constraints

- **Collections Allowed:** Unlike Assignment 2, you **may** use `java.util` classes (`ArrayList`, `HashMap`, `HashSet`, `LinkedList`, `PriorityQueue`, `Stack`, `Queue`, `Collections`, etc.) in this assignment. The focus is on graph algorithms, not on reimplementing data structures.
- **Provided Structure:** You will be given a `src` folder. You must only fill in the `// TODO` sections in the following files:
  - `Graph.java` // Part 1: Graph Data Structure
  - `GraphAlgorithms.java` // Part 2: Graph Algorithms
  - `SocialNetwork.java` // Part 3: Social Network Analysis
  - `Main.java` // used for testing (do not modify or submit)
- **Student Info:** Fill in `STUDENT_NAME`, `STUDENT_ID`, and `STUDENT_KU_EMAIL` at the top of **every** submitted file. The same information must appear in all files.
- **Extended Autograder:** You have a local autograder (`Main.java`). Passing these local tests is a good start but does not guarantee a full grade. Your final implementation grade will be determined by a more comprehensive grading.

Note: Any modification outside the `// TODO` sections will result in a grade of 0.

Part 1: Graph Data Structure [35 points]

File: `Graph.java`

Implement a weighted graph using an adjacency list representation. The graph stores its adjacency list as a `HashMap<Integer, ArrayList<Edge»`, where each key is a vertex and the associated list contains the outgoing edges from that vertex. The graph supports both directed and undirected modes, determined by the `directed` flag set at construction time.

A provided inner class `Edge` stores a destination vertex and a weight. The constructor, `isDirected()`, `vertexCount()`, `edgeCount()`, `getVertices()`, `getEdges()`, and `toString()` are provided and must not be modified.

You will need to implement the following methods:

- `void addVertex(int v)` **[3 points]:** Add a vertex to the graph. If the vertex already exists, do nothing. Otherwise, add it with an empty edge list.
- `boolean hasVertex(int v)` **[2 points]:** Return `true` if the graph contains the given vertex, `false` otherwise.
- `void addEdge(int src, int dest, int weight)` **[6 points]:** Add a weighted edge from `src` to `dest`. If `src` or `dest` do not exist, add them as vertices first. If the edge already exists, update its weight. For undirected graphs, also add (or update) the reverse edge from `dest` to `src`. Only increment `edgeCount` when adding a new edge (not when updating).
- `void addEdge(int src, int dest)` **[2 points]:** Add an unweighted edge from `src` to `dest` with a default weight of 1.
- `boolean hasEdge(int src, int dest)` **[3 points]:** Return `true` if there is an edge from `src` to `dest`. If `src` is not in the graph, return `false`.

- `int getWeight(int src, int dest)` **[3 points]:** Return the weight of the edge from `src` to `dest`. Return -1 if the edge does not exist or `src` is not in the graph.
- `boolean removeEdge(int src, int dest)` **[4 points]:** Remove the edge from `src` to `dest`. For undirected graphs, also remove the reverse edge. Return `true` if the edge was removed, `false` if it did not exist.
- `ArrayList<Integer> getNeighbors(int v)` **[4 points]:** Return a list of all neighbor vertex IDs reachable from `v` via outgoing edges. If `v` is not in the graph, return `null`.
- `int outDegree(int v)` **[2 points]:** Return the number of outgoing edges from vertex `v`. Return -1 if `v` is not in the graph.
- `int inDegree(int v)` **[2 points]:** Return the number of incoming edges to vertex `v`. For undirected graphs, this is the same as `outDegree(v)`. For directed graphs, count how many vertices have an edge to `v`. Return -1 if `v` is not in the graph.
- `void removeVertex(int v)` **[4 points]:** Remove a vertex and all edges incident to it. For directed graphs, remove all outgoing edges of `v` and all incoming edges to `v` from other vertices. For undirected graphs, remove the reverse edge from each neighbor. Update `edgeCount` accordingly.

Part 2: Graph Algorithms [65 points]

File: `GraphAlgorithms.java`

Implement several fundamental graph algorithms. All methods are static and operate on a `Graph` object from Part 1.

- `ArrayList<Integer> bfs(Graph g, int start)` **[12 points]:** Perform a Breadth-First Search traversal starting from the given vertex. BFS explores vertices level by level using a queue. Neighbors are visited in sorted (ascending) order to ensure a deterministic traversal. Return the vertices in visit order. If the start vertex is not in the graph, return an empty list.
- `ArrayList<Integer> dfs(Graph g, int start)` **[12 points]:** Perform a Depth-First Search traversal starting from the given vertex. This implementation uses an explicit stack. Neighbors are pushed in reverse sorted order so that they are popped in ascending order, ensuring a deterministic traversal. If a popped vertex has already been visited, skip it. Return the vertices in visit order. If the start vertex is not in the graph, return an empty list.
- `HashMap<Integer, Integer> dijkstra(Graph g, int start)` **[18 points]:** Compute the shortest path distances from a start vertex to all reachable vertices using Dijkstra's algorithm. Return a `HashMap` where each key is a reachable vertex and each value is the shortest distance from the start. Uses a `PriorityQueue<int[]>` of `{vertex, distance}` pairs ordered by distance. If a vertex has already been finalized, skip it. Only reachable vertices appear in the result. If the start vertex is not in the graph, return an empty map.

- `boolean hasCycle(Graph g)` **[10 points]:** Determine whether the graph contains a cycle. For directed graphs, use DFS-based cycle detection with three-color marking (WHITE/GRAY/BLACK). A cycle exists if DFS encounters a GRAY vertex (back edge). For undirected graphs, a cycle exists if DFS encounters a visited vertex that is not the immediate parent of the current vertex. You will need to write recursive helper methods.
- `ArrayList<Integer> topologicalSort(Graph g)` **[13 points]:** Compute a topological ordering of a directed acyclic graph (DAG) using Kahn's algorithm. Return `null` if the graph is undirected or contains a cycle. When multiple vertices have in-degree 0, pick the smallest vertex first (use a `PriorityQueue`). The result is an `ArrayList` of vertices in topological order.

Part 3: Social Network Analysis [20 points]

File: `SocialNetwork.java`

Apply graph algorithms to analyze a social network. Each vertex represents a person (identified by an integer ID), and each edge represents a friendship. All methods are static and operate on a `Graph` object from Part 1.

Important: For directed graphs, all methods in this section treat edges as undirected (i.e., if there is an edge from A to B, consider A and B as connected regardless of direction).

- `int countComponents(Graph g)` **[7 points]:** Count the number of connected components in the graph. A connected component is a maximal set of vertices with a path between every pair. Use BFS or DFS from each unvisited vertex. For directed graphs, consider both outgoing and incoming neighbors.
- `int mostConnected(Graph g)` **[5 points]:** Find the vertex with the highest total degree. For undirected graphs, total degree is `outDegree(v)`. For directed graphs, total degree is `outDegree(v) + inDegree(v)`. If multiple vertices tie, return the smallest vertex ID. Return -1 if the graph is empty.
- `int degreesOfSeparation(Graph g, int person1, int person2)` **[8 points]:** Compute the shortest path length (number of edges) between two people using BFS. Treat edges as undirected. Return 0 if `person1 == person2`. Return -1 if either person is not in the graph or there is no path between them.

Grading Breakdown

This homework is worth a total of **120 points**.

- Part 1: 35 points (Graph Data Structure)
- Part 2: 65 points (Graph Algorithms)
- Part 3: 20 points (Social Network Analysis)

Total: 120 points, will be normalized to 100.

Submissions

Upload the following to LearnHub as a **single ZIP** named `<ku_net_ID>_Assignment3.zip` (e.g., `sekmekcibasi23_Assignment3.zip`):

1. `Graph.java`
2. `GraphAlgorithms.java`
3. `SocialNetwork.java`
4. Your signed Academic Integrity Statement (`.pdf`, `.jpg`, or `.png`)

Place all files in a folder named `<ku_net_ID>_Assignment3/` inside the ZIP.

Note: `Main.java` is provided for you to test your code. Do **not** submit `Main.java`. Do **not** add package declarations. Do **not** rename files. Verify your ZIP structure before uploading.